

Assignment #6 - Trace It, Then Gate It

Syed Tabish Mobin | 9 September 2026

I used the handout's fictional claim with my existing Azure model and Phoenix for tracing. I kept the gate in plain Python middleware, which the brief allows. The payout only writes to a local SQLite ledger; no money moves.

1. Blast radius for my Milestone 2 agent

My S07 document agent, also used in S08, has two tools. I use these control rungs: R1 trace and restrict access; R2 validate and bound calls; R3 add durable idempotency for writes; R4 require exact-call human approval.

Tool	Classification and blast radius	Rung needed
file_search	Read: can expose documents the caller should not see.	R1 + R2: scoped corpus, access checks and call budget.
flag_for_human	Read/pure function today: returns a message, without creating a ticket.	R1 + R2: trace the call and validate the reason.
issue_payout (this lab, not Milestone 2)	Irreversible business action, simulated by a ledger insert.	R1-R4: exact approval plus a durable settlement key.

2. Where I would not put a gate

I would not ask someone to approve every `file_search` call. An analyst would keep clicking approve just to get an answer, so that would become rubber-stamping. I would enforce document permissions and a search budget instead. My current `flag_for_human` also does not need approval just to return a review message. If I turn it into a real ticket writer, I would treat that as a reversible write and add a stable request key.

3. Retry and idempotency

My Milestone 2 agent has no irreversible tool today. Repeating its search does not mutate the corpus; repeating the current review function does not create a ticket. For this lab's more dangerous payout, the local implementation is idempotent: `settlement:v1:C-2087` is unique in SQLite. The insert and approval consumption share one transaction. A retry after commit returns `already_paid`; a crash before commit leaves nothing paid. The claim's business ID survives a redeploy because it is stored outside the worker. All 11 tests passed, including concurrent retry and changed arguments. A real payment service would also need this key at the provider and reconciliation; SQLite alone cannot guarantee an external payment happens once.

4. What the trace added

The three ledgers showed A: one payout to AC-10045; B: no payout; C: one payout to AC-99999 after my approval was relayed into the terminal. B's final answer suggested checking compliance holds. Its approval-gate trace showed the actual reason: the changed account had no matching approval. The binding was `37d9ec4f...1c0b4288`, with `decision=blocked` and `allowed=false`. The screenshots show this blocked gate and the complete B trace. B made three model calls and used 1,172 tokens, details its final answer did not give. Across A/B/C I measured 3,439 tokens; the reference estimate was US\$0.001971, not an Azure invoice.

Each span answers a different question: the root shows the whole run; `model.chat` shows the model's inputs, outputs, latency and tokens; the read-tool span shows the account returned by the document; the payout-tool span shows the exact attempted settlement; the gate span shows whether that binding was allowed and, for C, who approved it.